



TESE510 - Atelier industrialisation des tests - TP Jenkins

Gaël Roustan (Argonauts)

2025-04-29

Abstract

Ce document contient les instructions pour mettre en application les principes d'une chaîne d'intégration continue.



Contents

TP - Intégration continue et métriques	3
Présentation	3
Jenkins	3
Objectif	3
Prérequis	3
Quelques informations utiles	3
Quelques informations utiles bis	4
Lot 1 Vue d'ensemble	4
Construire une image Docker à partir d'une image existante	5
Récupérer une image Docker	5
Fonctionnement du réseau sous Docker	5
Fichier de configuration pipeline Jenkins	6
Lot 2 Vue d'ensemble	7
Utiliser Docker dans Docker pour déclencher des tâches sur des agents Jenkins	7
Suite des opérations	9



TP - Intégration continue et métriques

Présentation

Au cours de ce TP, nous travaillons avec une solution d'Intégration Continue : Jenkins.

Jenkins

La solution Jenkins est une solution entièrement autonome qui permet d'automatiser toutes sortes de tâches. A travers le plugin Blueocean et l'utilisation d'un fichier de configuration `Jenkinsfile`, il est tout à fait possible de configurer une chaîne d'intégration complète sous forme de [Pipeline](#), à l'image de ce qui se fait avec Gitlab ou Github. Toute la configuration est incluse dans un unique fichier qui est versionnée avec le projet.

Les premiers pas avec Jenkins sous Docker sont décrits dans le fichier [README.md](#)

Pour déployer une première chaîne d'intégration continue autour d'une application Java, le tutorial [Build a Java app with Maven](#) est le point de départ incontournable.

Objectif

L'objectif de ce TP est de vous guider dans la mise en place d'une chaîne d'intégration continue complète avec le service Jenkins combiné à Sonar.

L'objectif de ce TP est de vous accompagner dans la mise en place d'une chaîne d'intégration continue avec Github (pour l'hébergement du code) et Jenkins.

Comme toujours, rien ne remplace la consultation de la documentation technique disponible pour chacune des solutions.

Prérequis

Il est nécessaire de récupérer le code source du projet Java fourni par l'équipe de Dev. Le projet est disponible sur un [drive non versionné](#).

Quelques informations utiles

Pour configurer ce projet dans une chaîne d'intégration, vous aurez besoin de l'outil maven et donc de la commande suivante pour lancer les tests d'intégration



```
mvn clean test
```

Si vous n'avez pas d'environnement prêt pour exécuter des tâches maven, aucun problème, il est également possible de passer par les conteneurs Docker. Attention à bien récupérer maven dans la version de Java attendue, à savoir la version 17.

```
docker run -v "$(pwd)":/app -w /app  
↳ maven:3.9.5-amazoncorretto-17 mvn clean install
```

Les tests d'intégration font appel à une base de données relationnelle. Pour l'utiliser dans la chaîne d'intégration vous devez indiquer au projet l'URL JDBC à utiliser. Cette URL est à rajouter dans la partie scripts du job :

```
export  
↳ DATABASE_URL_TEST=postgres://gael:@postgres:5432/restservice
```

Attention, le mot de passe vide n'est plus recommandé, il faut utiliser plutôt POSTGRES_HOST_AUTH_METHOD: trust

Vue d'ensemble Les principales étapes à suivre sont dans l'ordre

1. Télécharger le projet exemple Java
2. Initialiser 1 nouveau projet vide sur Gitlab
3. Déposer les sources du projet sur ce nouveau projet Gitlab
4. Configurer Jenkins pour lancer les tests uniquement sur les branches non main
5. Démarrer SonarQube
6. Configurer Jenkins pour déclencher l'analyse statique de code
7. Afficher sur la page d'accueil du projet les résultats de couverture de code - Badges

Quelques informations utiles bis

Pour configurer ce projet dans une chaîne d'intégration, vous aurez besoin de l'outil maven et donc de la commande suivante pour lancer les tests d'intégration

```
mvn clean test
```

Nous allons devoir créer notre propre image car Jenkins ne produit pour le moment pas d'image Docker avec la suite Blueocean comme précisé dans la [documentation](#)

Lot 1 Vue d'ensemble

Cette première série d'actions a pour objectif de réussir la compilation et les tests d'intégration de notre projet directement sur l'instance



maître de Jenkins.

1. Construire une unique image Jenkins avec maven avec pour alias DNS jenkins
2. Démarrer un conteneur Sonar avec pour alias DNS sonar
3. Vérifier que sonar est accessible depuis jenkins
4. Vérifier que jenkins est accessible depuis sonar
5. Créer une nouvelle tâche maven qui va envoyer l'analyse des sources sur sonar
6. Créer maintenant un pipeline et ajouter le fichier de configuration nécessaire sur le projet hébergé sur gitlab
7. La pipeline doit s'exécuter en local sur la machine Jenkins maître

Construire une image Docker à partir d'une image existante

Pour ajouter des outils, des dépendances, ou rendre persistante de la configuration à une image Docker existante, il est possible de créer sa propre image. Pour créer sa propre image, il faut créer un fichier Dockerfile et utiliser ensuite la commande

```
docker build -t name:tag .
```

Le fichier Dockerfile peut effectuer plusieurs actions comme par exemple définir une variable d'environnement, jouer des commandes shell, copier des fichiers. Pour en apprendre plus, consultez la [référence Dockerfile](#).

```
FROM jenkins/jenkins:lts
ENV JENKINS_SLAVE_AGENT_PORT 50001
```

Plus d'exemples de création de Dockerfile pour Jenkins sont disponibles sur le [référentiel officiel Github](#).

Récupérer une image Docker

L'un des annuaires officiels pour les images Docker est le [Dockerhub](#). A peu près n'importe quel éditeur a son image Docker déployée sur Dockerhub. Dans la suite du cours, vous verrez qu'enregistrer une image Docker pour la mettre à disposition ensuite n'est pas seulement réservé à Dockerhub. Il existe plusieurs solutions pour stocker en privé/public ses propres images Docker. Parmi les outils que nous utilisons, citons Gitlab et Nexus.

Fonctionnement du réseau sous Docker

Sur un même hôte Docker, il est possible de faire communiquer plusieurs conteneurs ensemble, et il est même possible de définir



des alias pour chacun de ces conteneurs. Il faut avant tout créer un réseau auquel vous allez rattacher vos conteneurs.

```
docker network create --driver bridge jenkins-network
```

Ensuite, pour démarrer un conteneur en le rattachant par défaut au réseau jenkins-network, utiliser la commande suivante :

```
docker run --name jenkins-docker --rm --detach
  --privileged --network jenkins-network
  --network-alias docker
```

Notez également dans la commande ci-dessus l'utilisation de l'option `--network-alias` pour spécifier l'alias qui permettra de contacter ce conteneur docker sur le réseau jenkins. Cette opération de rattachement n'est pas possible avec le réseau par défaut.

Il est possible également de rattacher un conteneur à un réseau après avoir démarré le conteneur en question.

```
docker network connect jenkins-network jenkins-docker
```

Plus de détails sur le fonctionnement du réseau Docker sur la [page officielle](#).

Fichier de configuration pipeline Jenkins

Jenkins peut également stocker la définition de la tâche à exécuter dans un fichier stocké au même endroit que les sources du projet. Par défaut, ce fichier est appelé Jenkinsfile et est situé à la racine du projet. Il supporte 2 formats différents : déclaratif et scripté. Voici un exemple de fichier qui consiste à lancer les tests d'intégration et indique à Jenkins où trouver les résultats des rapports de test

```
pipeline {
  agent any
  stages {
    stage('Integration test') {
      steps {
        sh 'mvn test'
        junit 'target/surefire-reports/*.xml'
      }
    }
  }
}
```

Pour plus de détails, pensez à consulter la documentation officielle sur [la gestion de Jenkinsfile](#).



Lot 2 Vue d'ensemble

Cette seconde série d'actions a pour objectif de déclencher les tâches sur un agent différent de l'instance maître.

1. Recréer une simple image docker jenkins avec blueocean et maven
2. Vérifier que vous êtes en mesure d'exécuter la tâche de compilation
3. Profitons maintenant de la possibilité d'utiliser un agent docker. Attention pour docker dans docker, il faut se baser sur le tutorial suivant et créer une seconde image qui fonctionnera de pair avec Jenkins puisqu'elle servira de démon Docker pour Jenkins dans Docker
4. Modifier le fichier de configuration du Pipeline pour profiter de l'agent Docker

Utiliser Docker dans Docker pour déclencher des tâches sur des agents Jenkins

Jenkins Docker a besoin d'un démon Docker pour instancier des agents.

Pour obtenir un environnement Jenkins avec un maître possédant la nouvelle interface Blue Ocean, il faut suivre les étapes :

- Démarrer un conteneur Docker qui va agir en tant que Démon Docker

```
docker run --name jenkins-docker --rm --detach \
--privileged --network jenkins --network-alias docker \
--env DOCKER_TLS_CERTDIR=/certs \
--volume jenkins-docker-certs:/certs/client \
--volume jenkins-data:/var/jenkins_home \
--publish 2376:2376 \
docker:dind --storage-driver overlay2
```

- Créer une image personnalisée avec Blue Ocean

```
FROM jenkins/jenkins:2.479.3-jdk17
USER root
RUN apt-get update && apt-get install -y lsb-release
RUN curl -fsSL \
  /usr/share/keyrings/docker-archive-keyring.asc \
  https://download.docker.com/linux/debian/gpg
RUN echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/docker-archive-keyring.asc] \
  \
```



```
https://download.docker.com/linux/debian \
$(lsb_release -cs) stable" >
└─ /etc/apt/sources.list.d/docker.list
RUN apt-get update && apt-get install -y docker-ce-cli
USER jenkins
RUN jenkins-plugin-cli --plugins "blueocean
└─ docker-workflow"

• Utiliser l'option build de la commande docker
  docker build -t myjenkins-blueocean:2.479.3-1 .

• Démarrer un conteneur Docker basée sur l'image personnalisée
docker run --name jenkins-blueocean --restart=on-failure
└─ --detach \
  --network jenkins --env DOCKER_HOST=tcp://docker:2376 \
  --env DOCKER_CERT_PATH=/certs/client --env
└─ DOCKER_TLS_VERIFY=1 \
  --publish 8080:8080 --publish 50000:50000 \
  --volume jenkins-data:/var/jenkins_home \
  --volume jenkins-docker-certs:/certs/client:ro \
  myjenkins-blue:2.479.3-1
```

Attention à ne pas effectuer ces opérations dans cygwin. Nous avons constaté des problèmes de récupération des certificats, ce qui bloque le démarrage du démon Docker.

Pour plus de détails, consultez la documentation officielle sur l'utilisation de [Jenkins dans l'univers Docker](#), [Hello World](#), [Maven Project](#).

Importer un projet vide sur Gitlab Pour initialiser un projet vide sur gitlab, vous pouvez effectuer les commandes suivantes dans un terminal

```
cd existing_folder
git init
git remote add origin
└─ git@gitlab.com:gael.roustan/test.git
git add .
git commit -m "Initial commit"
git push -u origin main
```

Configurer Jenkins pour lancer une tâche Jenkins adhère au principe Pipeline As Code, ce qui impose notamment de stocker au même endroit que les sources du projet la manière dont le projet doit être testé, l'intégration accomplie et la publication de l'application.



Toutes les tâches exécutées par Jenkins sont définies dans un fichier au format groovy nommé Jenkinsfile.

Ce fichier doit être stocké à la racine du projet. Dans l'exemple ci-dessus, nous définissons une tâche "unit_test" qui est constituée d'une seule ligne de commande.

Pour plus d'informations sur le format et les options offertes par l'intégration continue de Jenkins, rendez-vous sur [Doc Jenkins](#).

Restreindre le périmètre de lancement des tâches Il est possible de définir un périmètre et des conditions d'exécution pour les tâches.

Configurer Jenkins pour déclencher l'analyse statique de code

Pour déclencher une analyse statique de code sur le projet Java qui vous a été fourni, vous devez utiliser la commande maven mvn verify sonar:sonar -Dsonar.qualitygate.wait=true

Activer le service PostgreSQL dans la chaîne d'intégration

Les tests d'intégration font appel à une base de données relationnelle. Pour l'utiliser dans la chaîne d'intégration vous devez indiquer au projet l'URL JDBC à utiliser. Cette URL est à rajouter dans la partie scripts du job :

```
export  
DATABASE_URL_TEST=postgres://gaël:@postgres:5432/restservice
```

Attention, le mot de passe vide n'est plus recommandé, il faut utiliser plutôt POSTGRES_HOST_AUTH_METHOD: trust

Pour plus d'informations sur le fonctionnement de la base PostgreSQL en tant que service intégré dans Jenkins, vous pouvez consulter la documentation disponible [lien](#)

Attention, les développeurs se sont aperçus d'une erreur dans le code de leur application pour les tests d'intégration avec la base de données. Ils sont sympas, ils vous fournissent le patch à appliquer au projet. Le patch est disponible à cette [adresse](#).

La commande pour appliquer un patch est la suivante.

```
git apply tc-restservice-api-db.diff
```

Suite des opérations

Rajouter une fonctionnalité qui interroge une base de données.



Rajouter le test correspondant.

Utiliser TestContainer pour déployer temporairement une base de données pendant les tests.